

第1章 软件体系结构概论

软件危机：是指计算机软件的开发和维护过程中所遇到的一系列严重问题。

危机的表现：软件成本日益增长、开发进度难以控制、软件质量差、软件维护困难。

消除软件危机的方法：软件工程,软件工程三要素：方法、工具和过程。

成因：用户需求不明确、缺乏正确的理论指导、软件规模越来越大、软件复杂度越来越高。

软件重用：是指在两次或多次不同的软件开发过程中重复使用相同或相近元素的过程。软件元素包括程序代码、测试用例、设计文档、设计过程、需求分析文档甚至领域知识。

构件：是指语义完整、语法正确和有可用性价值的单位软件。

构件分类方法：关键字分类法、刻面分类法和超文本组织方法。

构件重用的过程：检索与提取构件、理解与评价构件、修改构件、构件组装。

构件重用内容：需求重用、设计重用、代码重用和组织机构重用

3. 掌握软件体系结构的定义、软件体系结构的意义及发展史、软件体系结构研究内容

软件体系结构定义：**software architecture**，是具有一定形式的结构化元素,即构件的集合,包括处理构件,数据构件和连结构件。

意义：1 体系结构是风险承担者进行交流的手段 2 是早期设计决策的体现

3 是可传递和可重用的模型

体现原因：1 明确了对系统实现的约束条件 2 决定了开发和维护组织的组织结构

3 制约着系统的质量属性 4 通过研究软件体系结构可能预测软件的质量

5 使推理和控制更改更简单 6 有助于循序渐进的原型设计 7 可以作为培训的基础

四个发展阶段:1 “无体系结构”设计阶段：以汇编语言进行小规模应用程序开发为特征

2 萌芽阶段:出现了程序结构设计主题,以控制流图和数据流图构成软件结构为特征

3 初期阶段：出现了从不同侧面描述系统的结构模型,以 UML 为典型代表

4 高级阶段：以描述系统的高层抽象结构为中心，不关心具体的建模细节，划分了体系结构模型与传统软件结构的界限，该阶段以 **Kruchten** 提出的 “4+1”模型为标志

第2章 软件体系结构建模

软件体系结构模型的种类：结构模型、框架模型、动态模型、过程模型和功能模型，最常用的是结构模型和动态模型。

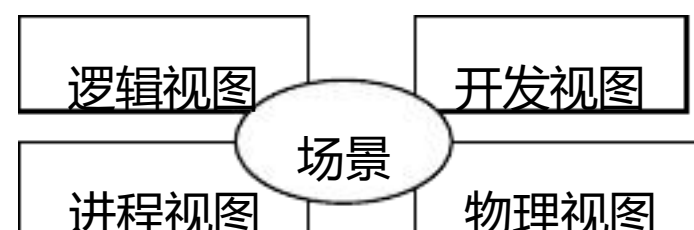
“**4+1**”模型：逻辑视图、进程视图、物理视图、开发视图和场景视图。

逻辑视图 **logic view**:最终用户：功能需求

开发视图：编程人员：软件管理

进程视图：系统集成人员：性能可扩充性、吞吐量等

物理视图：系统拓扑、安装、通信等



逻辑视图和开发视图描述系统的静态结构,而进程视图和物理视图描述系统的动态结构。

简答：描述模型和各模型的功能。

开发视图：也称为模块(**module**)视图，主要侧重于软件模块的组织和管理。

核心模型：构件、连接件、配置、端口和角色，其中，构件、连接件和配置是最基本的元素。

构件：是具有某种功能的可重用的软件模板单元，表示系统中主要的计算机元素和数据存储。连接件表示构件之间的交互;配置表示构件和连接件的拓扑逻辑和约束。

构件分为原子构件和复合构件.软件体系结构处于需求分析之后、总体设计之前。

软件体系结构的生命周期中各阶段任务

软件需求的类型包括：功能需求和非功能需求。

功能需求（开发的软件做什么）：为用户提供的服务、本系统为其他系统提供的服务

非功能需求：质量属性（系统在其生命周期过程中所表现出的各种特征）、约束（规定了开

发软件系统时必须遵守的限制条件).

情景（场景）：由人物、环境、事件、反应和结果等要素组成。

在软件开发中,借助场景说明用户对功能和质量的要求，对应为用例场景和质量场景。

场景：就是对某个实体与系统的一次交互的简要描述。

质量属性场景是一个有关质量属性的特定需求.质量属性场景由六部分组成：

刺激源:可以是风险承担者、计算机系统。

刺激：可以看作是一个事件。 环境：系统当前的状态。

制品：系统对事件作出反应的部分，可以是整个系统或系统的某一部分。

反应:事件到达后系统的相关行为。 反应度量：对反应结果提供某种形式的衡量。

第3章 软件体系结构风格（software architecture style）

软件体系结构风格：是描述某一特定应用领域中系统组织方式的惯用模式。

经典种类：数据流风格（批处理序列、管道和过滤器）、调用/返回风格（主程序和子程序、面向对象系统风格、层次结构）、独立构件风格（进程通信、事件系统）、虚拟机风格（解释器、基于规则的系统）。

C/S 结构包括：数据库服务器、客户应用程序和网络。

服务器任务：数据库安全性的要求；数据库访问并发性的控制;数据库前端的客户应用程序的全局数据完整性规则;数据库的备份与恢复。

客户应用程序任务：提供用户与数据库交互的界面;向数据库服务器提交用户请求并接收来自数据库服务器的信息；利用客户应用程序对存在于客户端的数据执行应用逻辑要求。

C/S 优点:1、C/S 体系结构具有强大的数据操作和事务处理能力,模型思想简单，易于人们理解和接受,一种分布式系统的模型。

2、系统的客户应用程序和服务构件分别运行在不同的计算机上,系统中每台服务器都可以适合各构件的要求，这对于硬件和软件的变化显示出极大的适应性和灵活性,而且易于对系统进行扩充和缩小，服务器符合较轻。

3 在 C/S 体系结构中，系统中的功能构件充分隔离，客户应用程序的开发集中于数据的显示和分析,而数据库服务器的开发则集中于数据的管理，不必在每一个新的应用程序中都要对一个 DBMS 进行编码.将大的应用处理任务分布到许多通过网络连接的低成本计算机上,以节约大量费用。

C/S 缺点：开发成本较高、客户端程序设计复杂、信息内容和形式单一、用户界面风格不一，使用繁杂,不利于推广使用、软件移植困难：不同平台的客户端需要不同版本客户端软件、软件维护和升级困难、新技术不能轻易应用。

三层 C/S 体系结构是将应用功能分成表示层、功能层和数据层三部分。P59 图 3—10

C/S 和 **B/S** 的异同：**B/S** 结构：浏览器、Web 服务器和数据库服务器

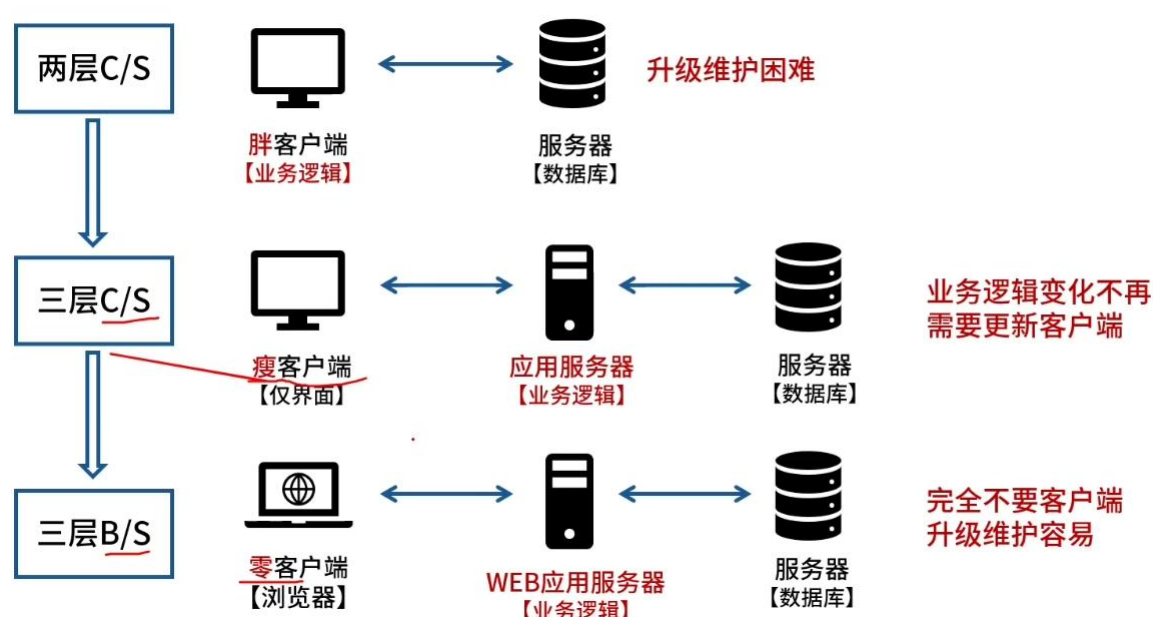
相同：B/S 和 C/S 都是架构，均是基于web 的服务。

不同点：1 B/S 是零客户端，C/S 则需要客户端安装软件。

2 B/S 体系结构缺乏对动态页面的支持能力，没有集成有效的数据库处理功能。

3 B/S 体系结构的扩展能力差，安全难以控制。

4 B/S 的数据提交一般以页面为单位，数据的动态交互性不强，不利用在线事务处理应用。



判断：一种软件体系结构只能有一种风格是错误的。

第4章 基于体系结构的软件开发

设计模式：是一些设计面向对象的软件开发的经验的总结。

智能体设计模式：RAG, A2A, MCP

组成部分：模式名称、问题、解决方案、后果

模式名称：用于描述模式的名字,说明模式的问题、解决方案和效果。

问题：说明在何种场合使用模式。

解决方案:描述设计的组成成分、它们之间的相互关系、各自的职责和合作方式。

后果：描述应用模式后的结果及使用模式应当权衡的问题。

MVC 设计模式:模型-视图—控制器,简述 MVC 组成。

设计模式的目录按照创建性、结构性和行为性三种目的划分为 23 种设计模式。

ABSD: architecture—based software design, 基于软件体系结构的设计方法。

ABSD 生命周期 (P203)

ABSDM 模型的过程：基于体系结构的软件开发模型，简称 **ABSDM**。过程划分为体系结构需求、设计、文档化、复审、实现、演化六个子过程。

基于软件体系结构的开发过程可以分为独立的两个阶段:实验原型阶段和演化开发阶段。

体系结构需求分析：从系统的质量目标、系统的商业目标和系统开发人员的商业目标分析。

文档：是在系统演化的每一个阶段，系统设计与开发人员的通信媒介，是为验证体系结构设计和提炼或修改这些设计所执行预先分析的基础。

文档的完整性和质量是软件体系结构成功的关键因素。

复审的目的是标识潜在的风险，及早发现体系结构设计中的缺陷和错误，包括体系结构能否满足需求、质量需求是否在设计中得到体现、层次是否清晰、构件的划分是否合理、文档表达是否明确、构件的设计是否满足功能和性能的要求等。

测试是最后一步,包括单个构件的功能性测试和被组装应用的整体功能和性能测试。

第5章 软件体系结构的分析与测试

可靠性：是软件系统应用或系统错误面前，在意外或错误使用的情况下维持软件系统的功能特性的基本能力。（可靠性是抽象的，但是可以计算其可靠性因子）

基于软件体系结构的软件测试与程序测试有所不同，它是检查软件设计的适用性，这种测试不考虑软件的实现代码，主要关注系统的结构和组装，保证软件的可靠性。

测试方法:测试内容、测试准则、测试需求和测试用例的生成

测试过程：单元测试、集成测试、系统测试。

严重性分析中失效方式的识别分为两种:单个构件的失效方式和单个连接件的失效方式。

严重性级别：灾难性的，危急的,边际性的和较小的。

体系结构主要关注系统的结构和组装。

基于体系结构描述的软件测试准则：测试应覆盖所有的构件及各个构件的接口、各个连接件的接口、构件之间的直接连接、构件之间的简介连接。

第6章 软件体系结构评估

体系结构评估中，一般采用刺激，环境，响应三方面来对场景进行描述。

SAAM和ATAM的概念。